

Response to Gnana Rajnan — Admin Dashboard to AI Pipeline Integration Clarifications

LocalAI TV — Admin Dashboard Plan v1.2

To: Gnana Rajnan (AI Pipeline Engineer) **From:** LocalAI TV Architecture Team **Subject:** Definitive answers to your 9 open questions; all recommendations adopted into Plan v1.2 **Date:** 14 May 2026 **Status:** All decisions locked

Note: This response is aligned with **Plan v1.2** which incorporates additional founder decisions: AI Pipeline migrates from Hostinger VPS to **AWS EC2 g4dn.xlarge (NVIDIA T4 GPU)** in Mumbai for 8-10× faster bulletin processing. Storage uses **AWS S3 (Mumbai)** with Cloudflare CDN for delivery (not R2). Scale target: 3,000 channels (pan-India) in 6 months. Your folder structure, retention policy, and integration contract recommendations remain adopted exactly — only the underlying storage tool changed from R2 → S3 (paths and lifecycle policy syntax updated accordingly).

Dear Gnana,

Thank you for the **Integration Clarifications** document. Your work is the most architecturally complete contribution to this design phase — your recommendations are essentially the production-grade specification we will now build to.

This response:

1. Confirms all your architectural recommendations are **adopted into Plan v1.1**.
2. Answers your **9 open questions** definitively.
3. Documents the contract changes triggered by your input.

Confirmation: Pipeline Architecture and Recommendations Adopted

Every recommendation in your document has been incorporated into Plan v1.1:

Your recommendation	Status	Section in v1.1
Admin Dashboard remains source of truth	✓ Adopted	§1 Architecture Overview
AI Pipeline processes only <code>status=approved</code> content	✓ Adopted	§4 Workflow + §15 Webhook
Webhook returns <code>202 Accepted</code> quickly; no inline AI/TTS/FFmpeg	✓ Adopted	§15
Idempotency key = <code>content_id + approved_version</code>	✓ Adopted	§15 + §7 schema
<code>processing_mode</code> flag (ai_pipeline / raw_video / notebooklm_generated)	✓ Adopted	§7 schema
Storage ownership rules (per folder)	✓ Adopted	§8
Bulletin output structure with timestamped folders	✓ Adopted exactly as proposed	§8
Retention policy (48hr bulletins, 24hr segments, 30d manifests)	✓ Adopted	§8 + §17a
Failure states (queued/processing/failed_retryable/failed_final/ready/published)	✓ Adopted exactly	§7 status enum
HMAC signature verification	✓ Adopted	§15 + §16
FastAPI intake + Celery + RabbitMQ + Redis architecture	✓ Confirmed (your existing stack)	§15
Build worker single-instance (FFmpeg + Windows file lock)	✓ Respected; Admin won't interfere	§15
Webhook adapter (not a trivial change — substantial integration work)	✓ Acknowledged; ~1 week effort estimate	§20
S3 lifecycle cleanup policy (translated from your R2 spec)	✓ Adopted	§8 + §17a
Maintenance worker for retries	✓ Aligned with our <code>pg_cron</code> watchdog	§17a

Your existing pipeline infrastructure is more mature than v1.0 of the plan assumed. This is excellent news.

Answers to Your 9 Open Questions

Q1: Can the same `content_id` be edited and approved again after a webhook was already sent?

Answer for v1: NO. Once approved and webhook sent, the `content_id` is locked.

Recovery path for post-approval corrections:

```
Super admin clicks "Cancel processing"
↓
DELETE /webhook/cancel-job/{job_id}
↓
Your pipeline:
- If task not picked up by Celery → remove from RabbitMQ → 200 OK
- If task in progress → Celery SIGTERM → 200 OK
- If task complete → 410 Gone
↓
Content reverts to status='pending' in Admin
Admin edits → re-approves → new webhook with approved_version=2
```

For Phase 2: Native re-approval flow with explicit supersession. Deferred until we see real usage need.

Q2: Will approved content include a version number?

Answer: YES. `approved_version` (integer).

```
{
  "content_id": "8a3f4b2c-9e1d-4f7a-b8c5-d6e7f8a9b0c1",
  "approved_version": 1,
  "idempotency_key": "8a3f4b2c-9e1d-4f7a-b8c5-d6e7f8a9b0c1_v1",
  ...
}
```

v1 rule: Always `1` (since we don't allow re-approval in v1). Schema is future-proofed for `2, 3, ...` in Phase 2.

Idempotency key format (per your suggestion):

- `idempotency_key = f"{content_id}_v{approved_version}"`
- For v1, every key is `..._v1`
- Phase 2: `_v2`, `_v3`, etc. for re-approvals

Q3: Is `content_id` globally unique across all constituencies, states, and content types?

Answer: YES — UUID v4 generated by Admin Dashboard at upload time.

```
Format: 8a3f4b2c-9e1d-4f7a-b8c5-d6e7f8a9b0c1
Entropy: 2^122 – practically infinite, never collides
Unique across:
• All states (AP, TG, Karnataka, Tamil Nadu, future expansions)
• All districts
• All constituencies
• All content categories (news, birthday, marriage, event, shopping, car, rental, job)
• All content sources (citizen, admin_direct, notebooklm_news, notebooklm_bulletin, noteb
```

You can safely use `content_id` alone as a partition key or filename prefix without geographic suffixes.

Q4: In modify+approve, will the webhook send modified content only?

Answer: YES. The webhook payload contains **ONLY** the final approved version. Original is preserved in Admin's audit log.

Example:

```
Original citizen submission (PRESERVED read-only in Admin's `content.original_data`):  
  headline:   "పై ఓవర్ ప్రారంభం హ్యాపీన్డ్"  
  description: "today flyover opened by CM" (English only, typo)  
  media:      [photo1.jpg, photo2.jpg, blurry_video.mp4]
```

After admin's modify+approve (THIS is what your pipeline receives):

```
  headline:   "కర్నూలు పై ఓవర్ ప్రారంభం"  
  description: "ఈరోజు సాయంత్రం సీఎం గారు ప్రారంభించారు"  
  media:      [photo1.jpg, photo2.jpg]
```

Your pipeline never sees the original — only the corrected version. The original remains in Admin's audit storage for dispute resolution and legal compliance (IT Rules 2021).

Q5: If the Admin Dashboard retries a webhook, will it resend the exact same body and signature?

Answer: YES — byte-for-byte identical body + same HMAC signature on every retry.

Implementation guarantees:

```
# Admin Dashboard webhook sender (pseudocode)  
def send_webhook(payload):  
    # JSON serialized with sort_keys=True for deterministic output  
    body_bytes = json.dumps(payload, sort_keys=True).encode('utf-8')  
  
    # HMAC signs the raw body bytes  
    signature = hmac.new(SHARED_SECRET, body_bytes, hashlib.sha256).hexdigest()  
  
    headers = {  
        'Content-Type': 'application/json',  
        'X-LocalAITV-Signature': signature,                # SAME on retry  
        'X-LocalAITV-Idempotency-Key': payload['idempotency_key'], # SAME on retry  
        'X-LocalAITV-Attempt': str(attempt_number),          # INCREMENTS (not signed)  
    }  
  
    response = requests.post(WEBHOOK_URL, data=body_bytes, headers=headers)
```

Key design choices:

- The signed body does NOT contain timestamps (would break idempotency)
- The attempt counter is in a separate header (not in signed body)
- The idempotency_key is part of the signed body (ensures retry consistency)

Your verification can rely on: same body bytes → same HMAC. Any retry that arrives with a different signature for the same idempotency_key indicates tampering — reject with 401.

Q6: Can already-approved content be edited and re-approved? (Similar to Q1)

Answer: Same as Q1 — NO for v1.

Use the cancel-job + new revision flow described in Q1. The `approved_version` field future-proofs for true re-approval in Phase 2 once we have data on real usage patterns.

Q7: Will webhook retries resend the exact same raw body?

Answer: YES — see Q5 for the detailed contract.

Your verification logic should:

- 1. Read the raw body bytes from the HTTP request (NOT re-serialized JSON)
- 2. Compute HMAC-SHA256 over those exact bytes
- 3. Compare to the `X-LocalAITV-Signature` header
- 4. Reject if mismatch

Important note: Some HTTP frameworks re-serialize JSON before exposing it to handlers. **Use the raw body**, not framework-deserialized JSON. In FastAPI this means using `await request.body()` directly.

Q8: Who owns thumbnail generation: Admin Dashboard or AI Pipeline?

Answer: SPLIT OWNERSHIP based on the use case.

Thumbnail	Generated by	When	Purpose
Photo thumbnail (for citizen-uploaded photos)	Admin Dashboard	On upload, via AWS Lambda + Cloudflare Images	Admin queue preview
Video preview frame (for citizen-uploaded videos)	Admin Dashboard	On upload, via FFmpeg single-frame extract	Admin queue preview
Final bulletin thumbnail (<code>thumbnail.jpg</code> in your folder structure)	AI Pipeline (you)	After bulletin assembly	Live broadcast preview
Individual processed item thumbnail	AI Pipeline (you)	After processing	App display

Rationale:

- Admin needs to SEE the citizen upload IMMEDIATELY for review — so Admin generates lightweight previews
- AI Pipeline produces the FINAL broadcast video — so AI generates the broadcast thumbnail (matches video output quality + branding)

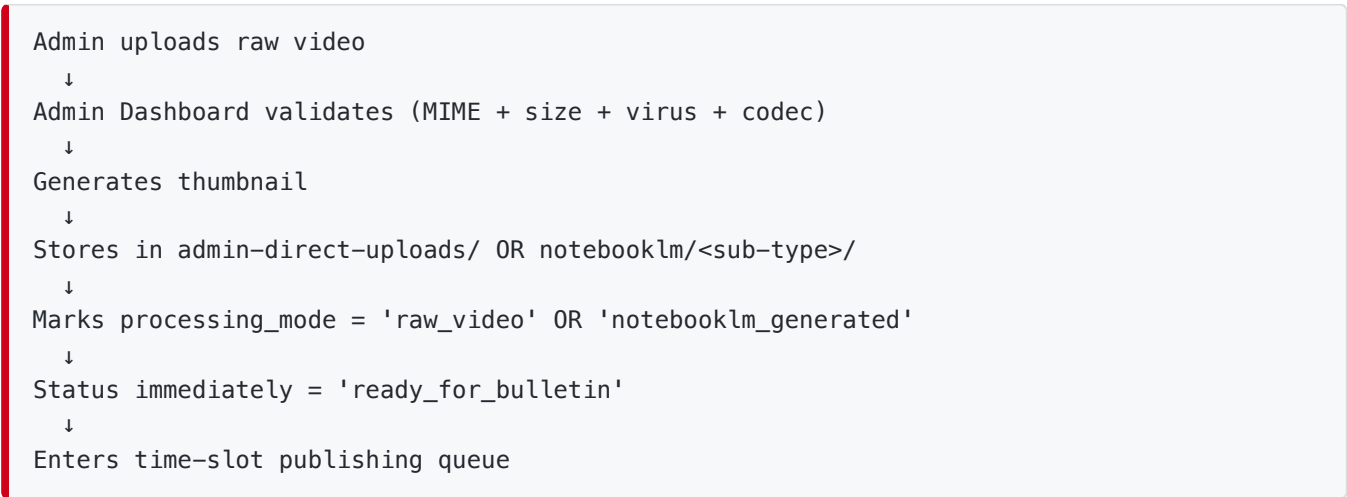
For your scope: Your existing `video_builder.py` should output `thumbnail.jpg` as part of the standard output bundle (per your proposed folder structure).

Q9: Should `admin-direct-uploads/` and `notebooklm/` be validated/transcoded before publishing?

Answer: Light validation YES, transcoding only as fallback. No TTS, no AI editing.

Validation step	admin-direct-uploads/	notebooklm/	Implemented in
MIME type check	✓	✓	Admin Dashboard (on upload)
File size limits (100 MB video, 10 MB photo, 200 MB bulletin)	✓	✓	Admin Dashboard
Virus scan (ClamAV via Edge Function)	✓	✓	Admin Dashboard
Codec validation (must be H.264/AAC playable)	✓	✓	Admin Dashboard
Auto-transcode if codec incompatible	⚠ Optional fallback	⚠ Optional fallback	Cloudflare Stream API
Thumbnail generation (1-frame extract)	✓	✓	Admin Dashboard via FFmpeg
AI script generation	✗ NO	✗ NO	n/a
Voice over (TTS)	✗ NO	✗ NO	n/a
FFmpeg editing	✗ NO	✗ NO	n/a

Direct-publish lane (no AI Pipeline involvement):



Your pipeline never receives a webhook for these. They bypass AI processing entirely. This matches your operational summary correctly.

One exception: If the build worker assembles bulletins that include both AI-processed items AND direct/notebooklm items, your bulletin assembler should be able to consume from any of these source folders (your existing `video_builder.py` likely already does this).

New Schema Additions in v1.1 Driven by Your Document

Content table (additions)

```
ALTER TABLE content ADD COLUMN approved_version INT DEFAULT 1;
ALTER TABLE content ADD COLUMN idempotency_key TEXT UNIQUE;
ALTER TABLE content ADD COLUMN processing_mode processing_mode_enum;
ALTER TABLE content ADD COLUMN target_channel TEXT;
ALTER TABLE content ADD COLUMN webhook_sent_at TIMESTAMPTZ;
ALTER TABLE content ADD COLUMN ai_job_id TEXT;
ALTER TABLE content ADD COLUMN ai_callback_at TIMESTAMPTZ;
ALTER TABLE content ADD COLUMN ai_output_url TEXT;
ALTER TABLE content ADD COLUMN ai_thumbnail_url TEXT;

CREATE TYPE processing_mode_enum AS ENUM (
    'ai_pipeline',
    'raw_video',
    'notebooklm_generated'
);

-- Status enum updated per your terminology
CREATE TYPE content_status AS ENUM (
    -- Admin workflow
    'pending',
    'in_review',
    'rejected',
    'escalated',

    -- AI processing pipeline (per your spec)
    'approved',           -- ready to send webhook
    'queued',             -- webhook accepted by AI Pipeline
    'processing',         -- Celery worker actively working
    'failed_retryable',   -- transient failure, will retry
    'failed_final',       -- permanent failure

    -- Publishing
    'ready_for_bulletin',  -- AI done OR direct upload ready
    'scheduled',          -- assigned to broadcast slot
    'published',          -- aired live

    -- Edge cases
    'cancelled',           -- super admin cancelled before processing
    'callback_lost',       -- AI processed but callback never received
    'archived'
);
```

New tracking tables

```
-- Idempotency + retry tracking
CREATE TABLE webhook_deliveries (
  id                UUID PRIMARY KEY,
  content_id        UUID REFERENCES content(id),
  idempotency_key    TEXT UNIQUE NOT NULL,
  payload           JSONB,
  attempt_count      INT DEFAULT 0,
  last_attempted_at  TIMESTAMPTZ,
  last_response_status INT,
  delivered_at       TIMESTAMPTZ,
  dead_lettered_at   TIMESTAMPTZ,
  created_at         TIMESTAMPTZ DEFAULT NOW()
);

-- Inbound callbacks from AI Pipeline
CREATE TABLE ai_callbacks (
  id                UUID PRIMARY KEY,
  content_id        UUID REFERENCES content(id),
  job_id            TEXT,
  idempotency_key    TEXT,
  status            TEXT,
  ai_output_url      TEXT,
  thumbnail_url      TEXT,
  duration_seconds   INT,
  error_message      TEXT,
  metrics           JSONB,
  received_at        TIMESTAMPTZ DEFAULT NOW()
);
```

S3 lifecycle configuration (translated from your R2 spec)

In v1.2, storage moved to **AWS S3 (Mumbai)**. The lifecycle policy translates directly — AWS provides richer storage classes (Standard / IA / Glacier IR / Deep Archive) for tiered cost reduction:

```
# AWS S3 lifecycle configuration (per your retention recommendations)
rules:
- id: bulletin-segments
  prefix: ai-processed-bulletins/*/segments/
  expiration:
    days: 1                                # 24 hours (DELETE)

- id: bulletin-final
  prefix: ai-processed-bulletins/*/final_bulletin.mp4
  expiration:
    days: 2                                # 48 hours (DELETE)

- id: bulletin-thumbnail
  prefix: ai-processed-bulletins/*/thumbnail.jpg
  expiration:
    days: 2                                # 48 hours (DELETE)

- id: bulletin-manifest
  prefix: ai-processed-bulletins/*/bulletin_manifest.json
  transitions:
    - days: 7
      storage_class: STANDARD_IA    # Move to cheaper tier
    - days: 30
      storage_class: GLACIER_IR     # Cheaper archival
  expiration:
    days: 365                        # Keep for 1 year audit

- id: bulletin-metadata
  prefix: ai-processed-bulletins/*/metadata.json
  transitions:
    - days: 30
      storage_class: GLACIER_IR
  expiration:
    days: 365

- id: temp-cleanup
  prefix: temp/
  expiration:
    days: 1                            # 24 hours

- id: ai-processed-tiering
  prefix: ai-processed/
  transitions:
    - days: 8
      storage_class: STANDARD_IA    # 50% cheaper
    - days: 31
      storage_class: GLACIER_IR     # 75% cheaper
    - days: 91
      storage_class: DEEP_ARCHIVE   # 95% cheaper (for compliance retention)
```

This achieves the same retention behavior as your R2 spec, but with **additional cost savings** because S3 has more storage tiers. The intent (delete bulletins after 48 hrs, keep manifests for audit) is preserved exactly.

Bulletin Output Structure (Your Proposal — Adopted Exactly)

```
ai-processed-bulletins/  
└─ <state>/  
    └─ <district>/  
        └─ <constituency>/  
            └─ bul_<YYYYMMDD_HHMMSS>/  
                ├── final_bulletin.mp4  
                ├── bulletin_manifest.json  
                ├── thumbnail.jpg  
                ├── segments/  
                │   ├── item_001.mp4  
                │   ├── item_002.mp4  
                │   └── item_003.mp4  
                └── metadata.json
```

Example:

```
ai-processed-bulletins/AP/Kurnool/Kurnool-Urban/bul_20260514_132000/final_bulletin.mp4
```

This structure supports:

- ☒ Constituency-level lookup
- ☒ Time-ordered retrieval (folder name is timestamp)
- ☒ Retention cleanup (lifecycle rules apply per folder pattern)
- ☒ Audit/replay (manifest + metadata kept longer)
- ☒ Future multi-channel publishing

Your Pipeline Readiness Assessment — Confirmed

Your "Already Aligned" list:

- ☒ FastAPI service layer
- ☒ Celery distributed tasks
- ☒ RabbitMQ queue buffering
- ☒ Redis result backend
- ☒ Media/AI/TTS/FFmpeg processing
- ☒ Build worker for bulletin generation
- ☒ Maintenance worker for retries

This means your AI Pipeline already addresses several of Abishek's production concerns:

- His #1 (webhook retry/idempotency) — your RabbitMQ + Celery handle this naturally
- His #2 (scheduler reliability) — your maintenance worker handles retries
- His #8 (queue infrastructure) — RabbitMQ is already there

Your "Still Required" list (we will work with you to scope these):

- `POST /webhook/content-approved` endpoint
- HMAC signature verification

- `content_id` /version idempotency check
- AWS S3 media URL download adapter (boto3 with signed URLs or direct IAM access — your AI Pipeline EC2 instance has S3 access via IAM role, simpler than R2 would have been)
- Output upload to `ai-processed/`
- Callback to Admin Dashboard
- Callback retry mechanism
- Direct-publish lane registration (admin/notebooklm)
- Cloud object lifecycle cleanup policy

Effort estimate: ~1 week of focused work. Your pipeline doesn't change; only the integration adapter is new.

Coordination Note

Three engineers have reviewed Plan v1.0:

- **Abishek** (full-stack) — production reliability concerns. Now addressed in v1.1 §17a/§17b/§16a.
- **Sameer** (AI engineer) — integration contract questions. Answered in dedicated response letter.
- **You** (AI engineer) — comprehensive architecture clarifications. This letter is your response.

Your contribution + Sameer's are complementary:

- **Sameer** defined what the webhook should do at the boundary.
- **You** specified how it integrates with the existing FastAPI + Celery + RabbitMQ infrastructure.

Together, you have made the AI Pipeline integration crisp and unambiguous.

Final Notes

Your document is the closest thing to a production specification we have received. v1.1 reflects every recommendation. The schema, folder structure, retention policy, failure states, and idempotency rules are now locked.

If you observe gaps or want to deep-dive any specific design choice — particularly the cancel-job flow, retry semantics, S3 lifecycle config, or the AWS EC2 GPU compute migration — please reply with specific points and we'll iterate.

v1.2 note for your team: The AI Pipeline runs on **AWS EC2 g4dn.xlarge in Mumbai (`ap-south-1`)** instead of Hostinger VPS. This solves the 15-minute bulletin processing time (drops to ~90 seconds with NVIDIA T4 + NVENC hardware encoding). Auto Scaling Group scales 3 → 50 workers based on RabbitMQ queue depth. Your existing code runs unchanged — just on a faster, more reliable instance.

With respect,

LocalAI TV Architecture Team LocalAI Media Network Pvt Ltd CIN: U63910KA2025PTC212593
Hyderabad, India

Plan v1.1 (separate document) reflects every change described above.